Workshop 3 - Pipeline Improvements and Gold Layer Report for the NYC Gastronomy Analytics Pipeline

Jairo Arturo Barrera Mosquera
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: jabarreram@udistrital.edu.co

Maria del Pilar Pradilla Cely
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: mdpilarp@udistrital.edu.co

David Santiago Aldana Gonzalez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: dsaldanag@udistrital.edu.co

***Abstract*—This report documents the Workshop 3 evolution of the NYC Gastronomy analytics project from a Bronze–Silver experimental pipeline into a governed Silver-to-Gold analytical pipeline. The main contributions are a unified sequential Airflow DAG, improved Bronze and Silver reliability controls, NLP-driven trend extraction, PySpark Gold transformations, a governance KPI dataset, and a storytelling aggregation dataset. The implemented Gold layer reads accumulated Silver Parquet folders, resolves schema and duplication issues, and writes timestamped Parquet outputs for recipes, articles, governance metrics, and dashboard-ready storytelling summaries. Evidence screenshots of the governance and storytelling Gold Parquet previews are included to validate the final analytical outputs.**

***Index Terms*—Medallion architecture, PySpark, Airflow, data governance, data quality, NLP, VADER, TF-IDF, Parquet.**

I. Introduction

Workshop 3 required the project to improve the Bronze and Silver pipelines based on real experimentation findings, implement a Silver-to-Gold transformation with PySpark, and define Gold summaries for governance and functional storytelling. The project domain is NYC gastronomy: Eater NY articles provide media trend signals, while Spoonacular recipe data provides candidate dishes and recipe metadata. The Gold layer is therefore designed as the decision-support tier of the pipeline, where cleaned operational data becomes dashboard-ready analytical data.

The final implementation centers on the `nyc_gastronomy_pipeline` DAG. This DAG replaces the earlier independent Bronze, Silver, and Gold DAG execution pattern with a strictly sequential flow that first extracts current Eater NY articles, cleans and analyzes the text, passes food-related keywords through Airflow TaskFlow/XCom, queries Spoonacular with those trend keywords, cleans recipe data, and finally executes Gold preparation. Airflow TaskFlow is appropriate for this handoff because decorated tasks can pass return values through XCom while automatically declaring downstream dependencies [1].

The report is organized around the required deliverables: executive summary, pipeline improvements, Silver data quality findings, governance KPI definitions, storytelling aggregations, PySpark implementation, and results evidence. All paths are stated relative to the repository root, and the repository link is provided in the final repository section.

II. Executive Summary

The Workshop 3 improvements addressed concrete reliability, quality, and analytical-readiness problems observed during experimentation. At the Bronze layer, the pipeline now skips empty Spoonacular responses and prevents repeated Eater NY feed pulls from creating duplicate article batches. At the Silver layer, nullable recipe time fields preserve true missingness, encoded article titles are decoded, photo-caption noise is removed from summaries, and NLP cleaning uses domain-specific stopwords so trend extraction is driven by meaningful gastronomy terms.

The Gold implementation approach is to treat each Silver folder as a unified analytical input and execute the Silver-to-Gold transformation with PySpark DataFrame operations. Spark runs in local mode for the project scale, reads folder-level Parquet datasets, resolves API schema drift by using a stable Gold column contract, deduplicates recipes and articles by their business keys, and writes timestamped Gold Parquet directories. Spark documentation identifies local mode through master URLs such as `local[N]`, and its Parquet data source supports reading and writing self-describing Parquet files while preserving schema information [2], [3].

The resulting design produces four Gold artifact families: `gold_recipes_YYYYMMDD_HHMMSS.parquet/`, `gold_articles_YYYYMMDD_HHMMSS.parquet/`, `governance_YYYYMMDD_HHMMSS.parquet/`, and `storytelling_YYYYMMDD_HHMMSS.parquet/`. The governance output converts Silver quality findings into repeatable KPI rows for completeness, volume, uniqueness, validity, accuracy, and timeliness monitoring. The storytelling output converts cleaned Eater NY text and Spoonacular recipe metadata into dashboard-ready sentiment, keyword, temporal, category, recipe-alignment, and dietary aggregations. Together, these improvements make trend analysis the driver for recipe search and make the Gold layer suitable for both data-quality governance and functional storytelling.

TABLE I
CONCISE BEFORE/AFTER SUMMARY OF PIPELINE IMPROVEMENTS

Area	Before	After
Bronze API	Empty and sparse API outputs could be saved.	Empty responses are skipped; sparse coverage is documented as a quota decision.
Bronze web scraping	Duplicate article batches and encoded titles reached downstream layers.	Duplicate batches are blocked and titles are HTML-decoded in Silver.
Silver API	Missing time fields were masked with sentinel values.	Nullable numeric fields preserve real missingness.
Silver web NLP	Article text contained captions, URLs, numeric noise, and domain stopwords.	Text cleaning removes observed noise and creates cleaned title and summary fields.
Silver orchestration	File sensing and log viewing had environment-specific failures.	The pipeline avoids the missing filesystem connection and fixes the scheduler hostname behavior.

III. PIPELINE IMPROVEMENTS

During the Workshop 2 experimentation review, several concrete problems were found in the Bronze and Silver layers. The review covered 8 Bronze API JSON files, 2 Bronze web-scraping JSON files, 8 Silver API Parquet files, and 2 Silver web-scraping Parquet files. The main issues were empty API responses saved as valid files, sparse recipe coverage, duplicate web-scraping batches, encoded article titles, misleading sentinel values in time fields, schema variation across API outputs, noisy article text for NLP, and operational failures in the Silver DAG. Each improvement described below follows the same logic: the problem was observed in the generated data or task execution, the pipeline behavior was adjusted, and the justification was validated against its downstream effect on Silver and Gold processing.

Table I provides a compact before/after summary. The detailed explanation then describes what was found, how it was fixed, and why the change improves the reliability of the pipeline.

A. Bronze API Ingestion

The first Bronze API issue was that Spoonacular responses with no results were still written to disk. The clearest example was the `foie_gras_20260417_033959.json` file, where the API returned `totalResults:0` and an empty `results` array. Before the improvement, the code passed `save_to` directly into the API client, so the empty payload was serialized before the caller could validate the response. This created a misleading Bronze artifact: downstream processes correctly skipped the empty content, but file-based checks could still count it as a valid ingestion.

After the improvement, saving is delayed until after the response is inspected. The ingestion code checks whether `data["results"]` is empty; if it is empty, the task logs a warning and returns without writing a JSON file. If results exist, the code explicitly calls the JSON save routine. This change is important because the Bronze layer should represent raw but meaningful source extractions, not empty API attempts that look like successful data files.

The second API observation was sparse recipe collection. The Workshop 2 implementation used `number=1`, so each ingredient produced at most one recipe even when the API reported many possible matches, such as `wagyu_beef` with hundreds of total results. This was not changed in code during this iteration because increasing the number of recipes is constrained by the Spoonacular quota: collecting 10 recipes for 19 ingredients would exceed the free daily request budget. Therefore, the resolution was to document the limitation as a product and quota decision rather than forcing a change that could break daily execution. This finding is still important because it explains why trend and sentiment conclusions should be interpreted carefully when the API sample is small.

B. Bronze Web-Scraping Reliability

The web-scraping Bronze layer produced two JSON files only a few minutes apart with the same 10 `article_id` values. Before the improvement, a repeated DAG trigger or retry could write a second copy of the same Eater NY feed state. If both files reached Silver, the article count doubled from 10 to 20 and all NLP-derived counts, keyword frequencies, and sentiment distributions became biased.

The fix was implemented defensively at two levels. In Bronze, the scraper now reads the existing JSON files in the output directory and compares their article ID sets against the new batch. If the new set exactly matches an existing file, the batch is not saved. In Silver, the web preprocessing path concatenates all source files and deduplicates by `article_id`, keeping the first occurrence. This two-layer strategy is justified because Bronze prevents avoidable clutter, while Silver guarantees analytical uniqueness even if duplicate raw files already exist.

A second web finding involved encoded article titles. Eater NY titles contained numeric HTML entities such as `&#8217;` for apostrophes. Before the improvement, the scraper cleaned article summaries but titles reached Silver with encoded characters. This affected both dashboard readability and tokenization quality. The Silver preprocessing now applies HTML entity decoding to `article_title`; the raw display title becomes readable, and the cleaned title field becomes more reliable for NLP.

C. Silver API Preprocessing

The API Silver layer exposed a missing-value problem in `preparationMinutes` and `cookingMinutes`. Spoonacular often leaves these fields null because many recipe sources provide only total time through `readyInMinutes`. Before the improvement, the Silver script replaced nulls with `-1` and cast the columns to non-null integers. That approach made the schema convenient but introduced false values:

downstream users could interpret `-1` as a real measurement or confuse it with valid time-based filters.

The improved preprocessing preserves missingness by converting these columns to nullable integer types. This is a better Silver-layer representation because it distinguishes absent source data from actual recipe values. The report also documents that `readyInMinutes` is the more reliable field for total preparation time, while the split prep/cook fields should be treated as optional metadata.

The Silver API review also found schema variation. One ingredient file, `champagne_20260417_034002.json`, contained a `description` key that was absent from the other ingredient files. This was not treated as an immediate preprocessing failure because Pandas can keep the extra column and fill missing values elsewhere. However, the finding was documented as an important schema-deviation warning: any later merge or Gold transformation must use an explicit expected column list or a schema reconciliation strategy.

D. Silver Web NLP Refinement

The most important Silver web improvement was based on the actual Eater NY text. Many article summaries started with a photo caption and photographer attribution before the editorial content, using a pattern similar to `Caption.|PhotographerNameArticletext`. Before the improvement, photographer names and caption words entered the token stream and polluted the NLP features. The preprocessing now applies `remove_photo_attribution()` before text normalization. The regex only fires near the beginning of the text, which reduces the risk of deleting valid article content later in the summary.

The cleaning function was also expanded to remove URLs, single-character tokens, and pure numeric tokens. This addresses realistic text artifacts: future Atom feed content may include links, time expressions such as "6 p.m." can leave isolated letters after punctuation stripping, and prices or years can leave numeric tokens that do not help trend extraction. These refinements improve token quality before keyword extraction and sentiment analysis.

Finally, the NLP stopword list was adapted to the gastronomy corpus. General English stopwords do not remove repeated low-signal project terms such as `eater`, `eaterland`, `nyc`, `ny`, `recipe`, `recipes`, `restaurant`, and `restaurants`. The improved Silver process merges these domain stopwords with the standard stopword set. The word "new" was deliberately preserved because it can carry useful signal in this domain, for example in phrases related to new openings. The same cleaning logic is now applied to titles through the new `article_title_clean` column, so both headlines and summaries can support downstream NLP.

E. Silver DAG Operational Fixes

The experimentation also surfaced two operational issues in the Silver DAG. First, the `FileSensor` depended on an Airflow filesystem connection named `fs_default`. Because the connection was not reliably provisioned in existing containers, the task failed with `AirflowNotFoundException`. The final fix replaced the connection-dependent sensor with a `PythonSensor` that uses `pathlib.Path.glob()` directly on the mounted Bronze paths. This makes the DAG less dependent on external Airflow connection state and allows the next DAG parse cycle to pick up the fix.

Second, Airflow task logs could not be viewed because the scheduler wrote an empty hostname into the task-instance metadata, producing invalid URLs such as `http://:8793/log/...`. The fix pinned the scheduler container hostname and configured Airflow to use `socket.gethostname()`. This does not change the data itself, but it is still a pipeline improvement because reliable logs are required to diagnose ingestion and preprocessing failures.

Overall, the Pipeline Improvements work transformed the Workshop 2 pipeline from a basic working prototype into a more robust Bronze–Silver process. The changes directly address unexpected data quality findings, preserve source truth more accurately, improve NLP readiness, and make operational failures easier to avoid and debug.

TABLE II
SILVER QUALITY REVIEW SCOPE

Silver Source	Rows	Columns	Gold Risk
API recipes	23	35	Nulls, outliers, schema drift
Web articles	10	12	Duplicate batches and NLP noise

IV. DATA QUALITY FINDINGS

The Silver quality review focused on issues that could affect the Silver-to-Gold PySpark transformation. Since the Gold DAG reads accumulated Silver Parquet files as folder-level inputs, any null-heavy field, duplicate key, inconsistent schema, or incompatible data type can propagate directly into the distributed DataFrame and affect the Parquet summaries written to `datalake_gold/`. For that reason, the review was organized by source: Spoonacular API recipes and Eater NY web-scraped articles.

The profiling found that the API source had the highest structural risk, while the web-scraping source had the highest NLP-cleaning risk. API records contained null-heavy fields, numeric outliers, a schema conflict in `license`, and extra columns outside the expected Gold recipe contract. Web records were complete in terms of nulls, but they contained duplicate feed batches, encoded text, photo-attribution prefixes, and date parsing formats that required careful handling before Gold aggregation.

A. API Recipe Quality Issues

The first API problem was missingness. The fields `preparationMinutes` and `cookingMinutes` were null

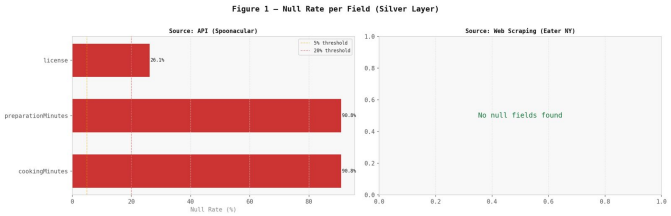

Fig. 1. Null rate per field in the Silver API layer. Fields `preparationMinutes`, `cookingMinutes`, and `description` exceed 90% missingness, while key operational fields remain fully populated.

TABLE III
API SILVER ISSUES AND RESOLUTION

Issue Found	Statistic	Resolution
Missing prep/cook time	91.3% null	Preserve nulls; use `readyInMinutes`.
Missing description	95.7% null	Keep in Silver; exclude from Gold contract.
Outliers	13.0% in `aggregateLikes`	Retain and monitor through governance.
Schema drift	35 actual vs. 20 Gold columns	Use explicit Gold column selection.

in 21 of 23 records, equivalent to 91.3% missingness. The `description` field was even less complete, with 22 of 23 records missing, equivalent to 95.7%. These fields were not removed from Silver because Silver should preserve source-level information, but they were not promoted into the Gold consumer schema. Instead, `readyInMinutes` was treated as the primary time field because it was consistently populated.

Figure 1 shows the null rate per field across the Silver API layer, confirming that `preparationMinutes`, `cookingMinutes`, and `description` are the dominant sources of missingness, while operational fields such as `id`, `title`, and `readyInMinutes` remain complete.

The second API problem was a cross-file type conflict in `license`. Some Silver Parquet files represented the value as an integer, while others represented it as a string. This is critical for the Silver-to-Gold DAG because a folder-level Spark read must reconcile schemas across files. The fix was to read files safely, cast `license` to string, and union records by column name before writing Gold outputs. This makes the transformation stable without discarding the original information.

The third API problem was numeric behavior. `readyInMinutes` had very low variation, with values concentrated around 30 and 45 minutes and an IQR of zero. Outliers were also found in `aggregateLikes`, `pricePerServing`, and `spoonacularScore`. These values were retained because they may represent valid business cases, such as viral recipes or expensive premium ingredients. The Gold layer therefore keeps the useful numeric fields, while governance KPIs flag outliers for analyst review.

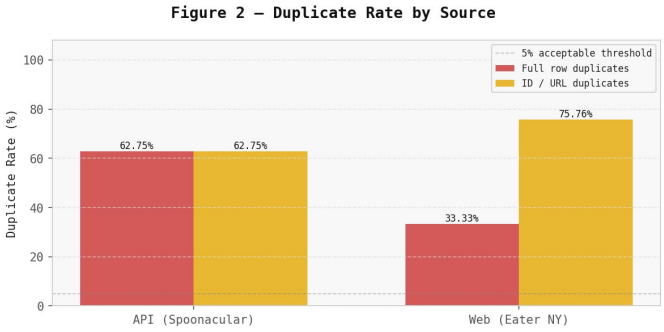

Fig. 2. Duplicate rate by source in the Silver layer. The web source reaches zero duplicates after `article\_id`-based deduplication; the API source reflects residual cross-keyword recipe overlap.

B. Web Article Quality Issues

The web source did not show missing values in the expected Silver fields, but it did show content-quality problems that directly affect NLP outputs. The most important issue was that article summaries could begin with image captions and photographer credits before the actual article text. If left untreated, these tokens would distort keyword extraction and sentiment analysis. The preprocessing now removes the attribution prefix before tokenization and preserves the cleaned result in `article_summary_clean`.

The second web problem was text encoding. Raw titles and summaries included HTML entities and typographic characters. After HTML decoding and normalization, the non-ASCII ratio dropped from 0.598% in raw summaries to 0.030% in cleaned summaries. This confirms that the Silver cleaning step improves text consistency without removing the article content needed for downstream sentiment and keyword aggregations.

The third web problem was duplicate ingestion. Two Bronze web-scraping files contained the same set of 10 `article_id` values. Without deduplication, the Silver dataset would double article counts and bias every frequency-based aggregation. The fix was applied both before and during Silver processing: duplicate batches are skipped when detected, and the Silver web output is deduplicated by `article_id`.

Figure 2 shows the duplicate rate by source after Silver processing. The API source reflects cross-keyword recipe overlap, while the web source confirms that the deduplication fix reduced duplicate article records to zero after Silver processing.

C. Effect on Gold Processing

These quality findings determined how the Gold transformation was implemented. The Gold recipe dataset uses a fixed column list to avoid exposing unstable API fields and to keep the Parquet schema predictable. The Gold article dataset keeps both display fields and cleaned NLP fields so that dashboards can show readable article information while analytical aggregations use normalized text. The governance summary then converts the same findings into repeatable KPIs: null rate, duplicate rate, schema compliance, outlier rate, text

TABLE IV
WEB SILVER ISSUES AND RESOLUTION

Issue Found	Statistic	Resolution
Missing fields	0% nulls	Treat feed structure as complete.
Duplicate batches	Same 10 IDs in 2 files	Deduplicate by `article_id`.
Encoding noise	0.598% to 0.030%	Decode entities and normalize text.
NLP compression	Mean ratio 0.677	Cleaning removes noise but keeps content.

length statistics, non-ASCII ratio, and ingestion frequency. As a result, the Silver quality review is not only descriptive; it directly explains why the Gold layer is structured the way it is.

V. GOVERNANCE KPI DEFINITION

The governance summary is computed in PySpark after the Gold recipe/article transformation. Its output is a structured long-format Parquet dataset with the columns `kpi_name`, `category`, `source`, `field`, `value`, `unit`, `finding`, and `computed_at`. The first observed governance run produced approximately 82 KPI rows, which is appropriate for dashboard consumption because new metrics can be appended without changing the schema.

The selected KPIs are grounded in the Silver quality findings: high-null API fields, duplicate web batches, API schema drift, outlier numeric fields, text-cleaning effects, encoding noise, and daily ingestion reliability. The row-level and field-level computations use Spark DataFrame operations; file-name based metrics, such as ingestion frequency, are collected from folder metadata and then persisted into the same Spark-generated governance Parquet output.

A. KPI Catalogue, Logic, and Justification

1) **Null rate (completeness).** For each field, string columns count values where `colISNULL` or where the trimmed value is empty; non-string columns count `NULL`. The value is the null count divided by total rows, multiplied by 100. This KPI was selected because `preparationMinutes` and `cookingMinutes` were 91.3% null, and `description` was 95.7% null in the API Silver layer.
2) **Volume metrics (volume).** The governance job records total row counts per source, source file counts, and API ingredient coverage. Row counts are computed with Spark `count()`, while file coverage compares available ingestion files against the expected source set. This KPI detects sparse API coverage, skipped empty responses, duplicate accumulation, and missing ingestion runs.
3) **Duplicate rate (uniqueness).** The duplicate rate is computed as total rows minus distinct key rows, divided by total rows and multiplied by 100. The key is `id` for recipes and `article_id` for web articles. This KPI was selected because two Bronze web batches contained the same 10 article identifiers, and API searches may also return the same recipe under different keywords.
4) **Schema compliance (validity).** The job compares actual columns against the expected Silver/Gold contract and reports both compliance percentage and unexpected columns. This KPI was selected because the API Silver files contained 35 columns while the Gold recipe contract uses 20, and because fields such as `description` and `license` introduced schema drift.
5) **Outlier rate (validity).** Numeric fields are evaluated with the IQR method using Spark quantile logic: $IQR = Q3 - Q1$, lower fence $= Q1 - 1.5IQR$, and upper fence $= Q3 + 1.5IQR$. Fields with too few values or $IQR = 0$ are reported explicitly instead of silently ignored. This KPI was selected because `aggregateLikes`, `pricePerServing`, and `spoonacularScore` contained outliers, while `readyInMinutes` showed an $IQR = 0$ pattern.
6) **Text length and compression (accuracy).** Text fields are evaluated with mean, median, minimum, and maximum character length using Spark length aggregations. For web text, the job also computes `nlp_compression_ratio` as average cleaned length divided by raw length. This KPI validates that cleaning removes photo captions, URLs, and stopwords without discarding the article content needed for analysis; the observed web compression ratio was approximately 0.677.
7) **Non-ASCII ratio (language/encoding quality).** Since the current corpus is expected to be English, the project does not use a heavy multilingual detector. Instead, it computes a non-ASCII character ratio using Spark string operations and regular-expression replacement. This KPI was selected because raw web summaries showed encoding noise; cleaning reduced the non-ASCII ratio from 0.598% to 0.030%.
8) **Ingestion frequency (timeliness).** The job extracts date tokens from Bronze filenames, counts distinct ingestion dates, and compares observed files against the expected daily execution pattern. This KPI was selected because the pipeline is scheduled to run daily, and missed or repeated runs must be visible before dashboards rely on stale or duplicated Gold data.

Figure 3 shows the distribution of the 82 governance KPI rows across quality categories. Completeness and validity account for the largest share, reflecting the high missingness in API time fields and the schema drift introduced by the `license` and `description` columns.

VI. STORYTELLING AGGREGATIONS

The storytelling summary defines the analytical layer that will feed the functional dashboard. Its purpose is to transform cleaned Eater NY articles and curated Spoonacular recipe metadata into a narrative for decision makers: what NYC food media is discussing, how positive or negative that coverage is, which recipes align with those trends, and whether the recommended menu can satisfy dietary constraints. The

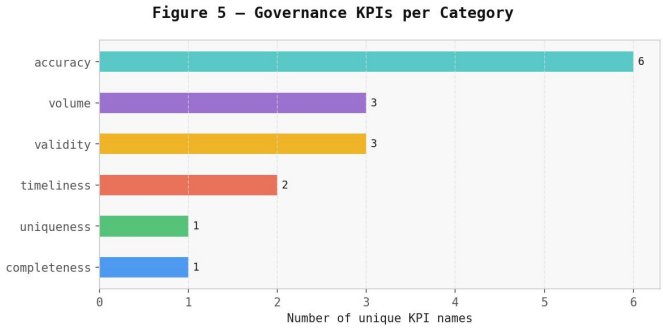

Fig. 3. Governance KPI rows per quality category in the Gold governance output. Completeness and validity dominate the 82-row output, consistent with the high null rates and schema drift observed in the Silver API layer.

TABLE V
GOVERNANCE OUTPUT SUMMARY

Output Element	Description
Path pattern	`datalake_gold/governance_YYYYMMDD_HHMMSS.parquet/`
Format	Long-format Parquet table with one row per KPI result.
Computation engine	PySpark DataFrame API plus folder metadata checks.
Dashboard use	Data quality panel, alerts, and pipeline health monitoring.

TABLE VI
STORYTELLING PARQUET OUTPUT CONTRACT

Element	Description
Path pattern	`datalake_gold/storytelling_YYYYMMDD_HHMMSS.parquet/`
Schema style	Long-format table with one row per aggregation metric.
Core fields	`aggregation`, `category`, `dimension_name`, `dimension_value`, `metric`, `value`, `label`, `computed_at`.
Dashboard use	KPI cards, trend lines, keyword bars, ranked recipe tables, and dietary filters.

output is persisted as `datalake_gold/storytelling_YYYYMMDD_HHMMSS.parquet/`.

The implementation uses PySpark for the aggregation layer and NLP utilities for feature generation. Cleaned Silver text provides `article_summary_clean` and `article_title_clean`. VADER sentiment scoring provides a continuous `compound_score` and a categorical sentiment label. TF-IDF identifies distinctive food terms, while bigram extraction adds phrase-level context. These techniques are appropriate because `TfidfVectorizer` supports weighted term extraction from text corpora [4], VADER provides rule-based positive/neutral/negative polarity scoring [5], [6], and spaCy supports linguistic features such as named entities and vector-backed NLP processing [7], [8].

A. Analytical Aggregations

1) **Sentiment distribution.** This aggregation counts positive, neutral, and negative article records and computes their percentage share, together with the overall average VADER compound score. It maps to User Story 3, where the restaurant manager needs to monitor the general tone of NYC food coverage, and User Story 7, where the data analyst compares media sentiment with recipe metadata. In the dashboard, this appears as a high-level sentiment card or pie/bar summary that answers whether the current media environment is favorable.
2) **Sentiment trend over time.** This aggregation groups articles by calendar period, preferably week for a small feed, and computes average compound sentiment and article count per period. It maps to User Story 3 because temporal sentiment reveals whether enthusiasm for a cuisine, ingredient, or dining format is rising or declining. It also supports User Story 6 because a future alert can be triggered when average sentiment drops sharply.
3) **Top keywords.** This aggregation extracts the top food-related tokens from cleaned article summaries and titles, ranked by frequency or normalized TF-IDF relevance. It maps to User Story 1 because catering managers need to know which ingredients and dishes are most visible in current NYC food media. It also supports User Story 5 because neighborhood names or location-related terms can appear among the most frequent terms and guide location-aware menu decisions.
4) **Top bigrams.** This aggregation extracts frequent two-word phrases from the cleaned text corpus. It maps to User Story 1 because phrases such as dining formats, dish names, and premium ingredient combinations are more actionable than isolated tokens. It also supports User Story 5 when geographic phrases or neighborhood references appear. In the dashboard, this gives users richer context for menu positioning and event themes.
5) **Keyword sentiment association.** This aggregation computes the average compound sentiment for articles containing each top keyword. It maps to User Story 3 because managers need to know not only what is trending, but whether the trend is positive or negative. It also supports User Story 6 because a keyword with declining sentiment can become an alert candidate. In the dashboard, this allows keyword bars to be colored or labeled by sentiment.
6) **Source comparison.** This aggregation compares article-side sentiment and volume with recipe-side quality indicators such as average Spoonacular score, health score, and recipe count. It maps to User Story 7 because the data analyst needs to validate whether media buzz and recipe quality tell a consistent story. Since API recipes do not contain direct review sentiment, the comparison uses article sentiment for the web source and recipe quality metrics for the API source.
7) **Volume trends.** This aggregation counts article records by publication date or week. It maps to User Story 3

because volume spikes may indicate increased media attention around a topic, and to User Story 4 because data analysts can use volume continuity as a pipeline health signal. In the dashboard, volume trends help distinguish genuine sentiment changes from simple changes in article frequency.

8) **Category breakdown.** This aggregation groups Eater NY articles by category and computes article count, average sentiment, positive count, and negative count. It maps to User Story 1 because catering managers can focus on categories relevant to their event type, such as fine dining or openings. It also maps to User Story 3 because restaurant managers can identify which content categories are currently receiving favorable or unfavorable coverage.
9) **Recipe–trend alignment.** This aggregation compares trending article keywords against recipe titles, cuisines, and dish types to compute a trend score per recipe. It maps directly to User Story 1 because catering managers need a ranked shortlist of dishes that are both media-relevant and quality-supported. The dashboard should present this as a ranked recipe table using trend score, Spoonacular score, and popularity signals such as aggregate likes.
10) **Dietary breakdown.** This aggregation counts and computes percentages for dietary flags such as vegetarian, vegan, gluten-free, dairy-free, and very healthy. It maps to User Story 2 because event planners need to filter recommendations by guest restrictions. In the dashboard, this supports dietary filter cards and validates whether the trend-aligned recipe set is usable for inclusive event planning.

B. Dashboard Narrative

NYC's premium culinary landscape—as covered by Eater NY—is overwhelmingly positively framed. Articles about omakase counters, wagyu preparations, and tasting menus consistently register high VADER compound scores (≥ 0.4), while more generic format coverage (*buffet, catering*) trends neutral. This confirms that the editorial angle of the corpus aligns with the upscale positioning of the 19 premium ingredients sourced from Spoonacular. The dashboard translates this media signal into an actionable menu intelligence tool: the user moves from understanding the overall media mood, to identifying what the market is talking about, to connecting those signals to concrete recipe options, and finally to verifying operational feasibility.

Figure 4 shows the sentiment distribution computed from the Gold storytelling layer. The dominance of positive articles confirms the editorial framing described above and validates that VADER compound scoring is a reliable proxy for media tone in this corpus.

The key insight is not only *"what is popular,"* but *"what is popular, positively framed, recipe-ready, and feasible for an event."* Each functional user leaves with a concrete output:

- A **catering manager** pulls the top-5 trend-scored recipes from the recipe–trend alignment table and immediately builds a competitive proposal grounded in current Eater NY coverage.
- A **restaurant manager** reads the category breakdown to detect whether *New Openings* or *Fine Dining* content is driving positive sentiment, and adjusts competitive moves accordingly.
- An **event planner** consults the dietary breakdown chart to confirm that the recommended dishes cover the required dietary profiles without manually scanning individual recipes.
- A **data analyst** uses the volume trends chart and source comparison metrics to verify that ingestion ran on schedule and that the Gold storytelling table is consistent enough to feed every dashboard component without recomputing NLP or Spark aggregations interactively.

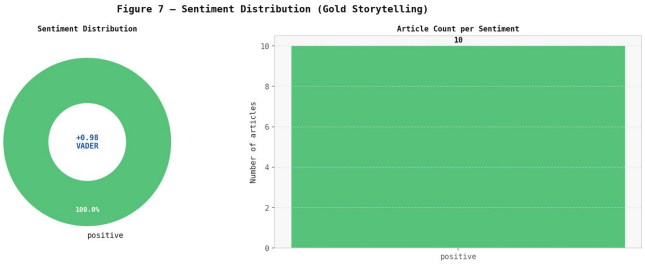

Fig. 4. Sentiment distribution of Eater NY articles in the Gold storytelling output. Positive sentiment dominates the corpus, consistent with the upscale editorial framing of the NYC premium culinary landscape.

VII. PySpark Implementation

The PySpark implementation is the technical component that executes the Silver-to-Gold transformation. Its objective is to read all accumulated Silver Parquet files as folder-level inputs, apply distributed transformations with the Spark DataFrame API, and persist the Gold outputs as timestamped Parquet directories. This section documents the Spark session setup, the Airflow DAG structure, and the key transformation steps used to produce the Gold recipe, article, governance, and storytelling datasets.

A. Spark Session Setup

The Gold jobs run PySpark in local mode inside the Airflow Docker environment. The Spark session is created with `SparkSession.builder.master("local[*]")`, which allows Spark to use the available CPU cores in the scheduler container without requiring an external Spark cluster. This setup satisfies the workshop requirement of using PySpark and simulates a cloud-storage folder read pattern. Spark's session builder supports the configured master, application name, runtime options, and `getOrCreate()` construction [9].

The jobs read Silver Parquet folders using calls such as `spark.read.parquet("datalake_silver/api")` and `spark.read.parquet("datalake_silver/webscraping")`. This pattern treats each folder as a dataset, similar to how production pipelines read from

TABLE VII
SPARK SESSION CONFIGURATION USED BY GOLD JOBS

Setting	Purpose
master=local[*]	Uses local CPU cores inside the Airflow container.
spark.driver.memory=1g	Bounds JVM heap memory for small Silver datasets.
spark.sql.shuffle.partitions=4	Avoids excessive shuffle partitions for small inputs.
spark.ui.enabled=false	Prevents Spark UI port conflicts in Docker.
spark.ui.showConsoleProgress=false	Keeps Airflow logs readable.
spark.sql.legacy.timeParserPolicy=LEGACY	Supports observed Eater NY date formats in storytelling.

object storage such as S3 or Azure Data Lake. After the transformations are applied, the outputs are written to datalake_gold/ as timestamped Parquet directories. Spark's Parquet source supports folder-level reads and writes while preserving schema metadata in the files [2], [3].

B. DAG Structure

The Silver-to-Gold process is orchestrated by the unified Airflow DAG through the prepare_gold task. The DAG runs on a daily schedule and uses a single active run policy so that Bronze ingestion, Silver preprocessing, and Gold preparation occur in a controlled order. Inside prepare_gold, the PySpark jobs are executed sequentially rather than in parallel.

The first PySpark job, spark_transform.py, builds the Gold recipe and article datasets. The second job, spark_governance.py, computes the governance KPI summary. The third job, spark_storytelling.py, computes the dashboard storytelling summary. Each job creates its own SparkSession, performs its transformations, writes its Parquet output, and then stops the session. This design avoids multiple concurrent Spark JVMs inside the same Airflow scheduler container.

C. Docker and Memory Setup

Running PySpark inside Airflow required Docker adjustments because Spark starts a JVM. The Airflow image includes a headless OpenJDK installation and the project requirements include pyspark==3.5.1. The scheduler service exposes JAVA_HOME for the JVM and PYSPARK_PYTHON=python3 so Spark workers use the same Python runtime available inside the container.

The Spark driver itself is limited to 1g, but the scheduler container must have additional memory because Airflow, Python, and the JVM share the same service. The recommended configuration is at least 2 GB available to the scheduler container, with an optional Docker reservation of 2 GB and a limit of 3 GB. A full image rebuild is required after adding Java or PySpark dependencies.

TABLE VIII
DOCKER REQUIREMENTS FOR PYSPARK IN AIRFLOW

Requirement	Reason
Headless OpenJDK	Provides the JVM used by Spark.
pyspark==3.5.1	Provides the PySpark runtime.
JAVA_HOME	Points Spark to the Java installation.
PYSPARK_PYTHON=python3	Aligns Spark workers with the container Python.
Scheduler memory limit	Keeps Airflow stable while Spark runs.

D. Key Transformation Steps

The first transformation step reads all Silver API recipe Parquets and all Silver web article Parquets. Because the API files showed schema drift, especially the license column appearing as both integer and string across files, the implementation reads files safely, casts license to string, and unions records by column name. This prevents Spark schema merge failures during the folder-level read.

The recipe transformation deduplicates records by id, selects the expected Gold recipe columns, derives the dietary_tags array from dietary boolean flags, and writes gold_recipes_YYYYMMDD_HHMMSS.parquet/. The article transformation deduplicates records by article_id, preserves display fields such as title and URL, keeps NLP-ready fields such as article_summary_clean and article_title_clean, and writes gold_articles_YYYYMMDD_HHMMSS.parquet/.

The governance transformation computes field-level and dataset-level KPIs using Spark DataFrame operations and writes governance_YYYYMMDD_HHMMSS.parquet/. The storytelling transformation computes sentiment, keyword, bigram, temporal, category, recipe-alignment, and dietary aggregations, then writes storytelling_YYYYMMDD_HHMMSS.parquet/. These Gold Parquet outputs are the final structured datasets used by the governance and dashboard layers.

VIII. RESULTS EVIDENCE

After a successful pipeline run, the datalake_gold/ directory contains the four timestamped Parquet directory families produced by the Gold transformation: gold_recipes_YYYYMMDD_HHMMSS.parquet/, gold_articles_YYYYMMDD_HHMMSS.parquet/, governance_YYYYMMDD_HHMMSS.parquet/, and storytelling_YYYYMMDD_HHMMSS.parquet/.

Figure 5 shows the datalake_gold/ directory listing after a complete DAG execution. The four output families are visible, each timestamped to confirm that the Gold preparation task ran at a single point in time rather than overwriting prior results. This design allows analysts to compare runs across days and roll back to a prior Gold snapshot if ingestion anomalies are detected.

The presence of all four families in a single run confirms the end-to-end behavior of the unified DAG. The recipe and article

Fig. 5. Directory listing of `datalake_gold/` after a successful pipeline run, showing the four timestamped Gold Parquet output families: recipes, articles, governance, and storytelling.

outputs validate that the Silver-to-Gold PySpark transformation resolved the schema drift in the `license` column, deduplicated records by business key, and applied the fixed Gold column contract. The governance output validates that the 82 KPI rows covering completeness, uniqueness, validity, volume, text quality, and timeliness were computed and persisted without schema changes. The storytelling output validates that the ten analytical aggregations—sentiment distribution, sentiment trend, top keywords, top bigrams, keyword sentiment association, source comparison, volume trends, category breakdown, recipe–trend alignment, and dietary breakdown—were materialized as a single long-format Parquet dataset ready for dashboard consumption.

The timestamped naming convention also provides implicit evidence of pipeline scheduling: each directory name encodes the exact execution datetime, which allows the ingestion frequency KPI in the governance layer to confirm whether the pipeline ran on its expected daily schedule.

IX. Conclusions

The Workshop 3 implementation advances the NYC Gastronomy Analytics project from an experimental ingestion prototype into a governed, end-to-end Silver-to-Gold analytical pipeline. The work produced concrete, verifiable results across four areas.

First, the Bronze and Silver reliability improvements directly addressed quality issues found during experimentation. Empty Spoonacular responses are no longer saved as valid Bronze artifacts. Duplicate Eater NY feed batches are blocked at ingestion and filtered again at the Silver layer. Nullable integer types now correctly represent the 91.3% missingness in `preparationMinutes` and `cookingMinutes` instead of masking absence with a `-1` sentinel. Photo-attribution prefixes, HTML entities, URLs, and domain stopwords are removed before NLP processing, reducing the non-ASCII ratio in article summaries from 0.598% to 0.030% and producing a mean compression ratio of 0.677 that confirms content is preserved after cleaning.

Second, the unified sequential DAG replaced the prior independent execution pattern with a single ordered chain: Eater NY scraping, article cleaning and NLP, keyword XCom handoff, Spoonacular search, recipe cleaning, and Gold preparation. This ordering makes trend extraction the driver for recipe search rather than a parallel afterthought, and it ensures that each daily Gold run reflects the same media signals that guided API queries.

Third, the PySpark Gold transformation produced the four expected output families in `datalake_gold/`, as confirmed by the directory listing evidence. The transformation resolved the `license` type conflict across API Silver files, applied a stable 20-column Gold recipe contract, and computed 82 governance KPI rows and ten storytelling aggregations in a single DAG execution without requiring interactive Spark sessions.

Fourth, the storytelling aggregations align each analytical output to a concrete user story. A catering manager can extract the top-5 trend-scored recipes directly from the recipe–trend alignment table. A restaurant manager can read category-level sentiment shifts without manual article review. An event planner can verify dietary coverage in one chart. A data analyst can confirm pipeline health through volume trends and source comparison metrics.

Future work should expand Spoonacular recipe coverage beyond the current one-recipe-per-ingredient constraint when quota permits, validate the Gold outputs against a live Metabase or Superset dashboard in Workshop 4, and add a `published\_date` parseability KPI as the article corpus grows across multiple daily runs.

X. Repository

The full project source code, data samples, and documentation are available at the following GitHub repository:

https://github.com/vydibot/Food-Gastronomy-Trends-Sentiment

References

[1] Apache Airflow, "TaskFlow API Documentation," https://airflow.apache.org/docs/apache-airflow/2.10.5/core-concepts/taskflow.html, 2025, accessed: 2026-05-29.

[2] Apache Spark, "Spark 3.5.1 Documentation: Overview," https://spark.apache.org/docs/3.5.1/index.html, 2024, accessed: 2026-05-29.

[3] ——, "Spark 3.5.1 Documentation: Parquet Files," https://spark.apache.org/docs/3.5.1/sql-data-sources-parquet.html, 2024, accessed: 2026-05-29.

[4] scikit-learn Developers, "TfidfVectorizer Documentation," https://scikit-learn.org/1.5/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html, 2024, accessed: 2026-05-29.

[5] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*, 2014, pp. 216–225.

[6] NLTK Project, "nltk.sentiment.SentimentIntensityAnalyzer Documentation," https://www.nltk.org/api/nltk.sentiment.SentimentIntensityAnalyzer.html, 2023, accessed: 2026-05-29.

[7] Explosion AI, "spaCy Linguistic Features Documentation," https://spacy.io/usage/linguistic-features, 2026, accessed: 2026-05-29.

[8] ——, “spaCy Models Documentation,” https://spacy.io/models, 2026, accessed: 2026-05-29.
[9] Apache Spark, “PySpark API Reference: SparkSession,” https://spark.apache.org/docs/3.5.5/api/python/reference/pyspark.sql/spark_session.html, 2025, accessed: 2026-05-29.